Mock 1 Easy 02 | Delta Inventory - UPDATE, History and Time Travel

SUBMISSION TEMPLATE

Difficulty	Marks	Tasks	Working budget
Easy	10	4	17 minutes (Easy)

Objective

Create a managed Delta inventory table, change one stock value and verify the previous version.

Evidence rules - read before starting

There is NO one-task/one-screenshot mandate and NO maximum screenshot count. Provide as many clear taskwise images as the rubric requires. If both implementation and result are assessed, include BOTH. A long SQL/PySpark/pipeline definition may be split into Code_A, Code_B and Code_C screenshots, plus separate Result_A / Result_B screenshots. Images must be legible, not tiny stitched collages. A UI/configuration screenshot is acceptable when the task is menu-based. Never rerun a mutation only to capture evidence; use the original executed code and saved result. The evaluator awards marks only from the submitted visible evidence; merely supplying a workspace link is not a substitute.

Environment and required names

Use your existing assigned Unity Catalog. Reuse schema mock1 for all four questions. Create the schema if missing; do not create a catalog, storage credential, external location, or an unrelated warehouse. Each question creates ONLY its own EXTERNAL volume using the instructor-supplied complete ADLS URI beneath the approved external location. Upload source CSVs inside input/. Managed tables and SDP output must be in schema-managed Delta storage, not inside volume files.

Item	Required value
Catalog	Existing assigned trainee catalog
Schema	mock1
Question external volume	m1_e02_files
Assigned relative ADLS path	mock1/e02/files/ under trainee root
Input directory	/Volumes/<trainee_catalog>/mock1/m1_e02_files/input/
Required saved object(s)	m1_e02_inventory

Key terms and formulas

- Delta history records table operations and associated version numbers.
- Time travel reads an earlier Delta table version without changing current records.

Dataset description

File(s): inventory.csv. Header present; use the supplied data types. Blank values, if any, represent missing values. Do not rename files.

Column	Type	Meaning
product_id	STRING	Unique product
product_name	STRING	Product description
stock_qty	INTEGER	Available stock

Trainee: _____Soumiya Ayyappan_____ Candidate ID: _____ Catalog: _____

Date: _____ Notebook / SQL editor / pipeline / dashboard link: _____

Task requirements and screenshot placeholders

Task 1. Load inventory as Delta | 3 marks

A. Setup UI: EXTERNAL volume location and filename.

B. Code: typed load and managed Delta table creation.

C. Result: all four starting quantities and baseline version.

Use descriptive filenames such as M1_E02_T01_Code_A.png, M1_E02_T01_Result_A.png, or M1_E02_T01_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Setup UI: _____



Paste screenshot(s) - Code: _____

```python
# Define the schema
schema = StructType([
    StructField("product_id", StringType(), False),
    StructField("product_name", StringType(), True),
    StructField("stock_qty", IntegerType(), True)
])

# Load data from volume
df = (spark.read
    .format("csv")
    .option("header", True)
    .schema(schema)
    .load("/Volumes/cat_ltm10877547/mock1/m1_e02_files/input/E02_inventory.csv")
)

# Write as managed Delta table
df.write.format("delta").mode("overwrite").saveAsTable("products")

# Show all rows and baseline version
spark.sql("SELECT * FROM products").show()
```

Paste screenshot(s) - Result: _____

```
df: pyspark.sql.connect.dataframe.DataFr
+----------+------------+---------+
|product_id|product_name|stock_qty|
+----------+------------+---------+
|      P101|    Keyboard|       20|
|      P102|       Mouse|       12|
|      P103|     Monitor|        8|
|      P104|     USB Hub|       15|
+----------+------------+---------+

+-------+
|version|
+-------+
|      1|
+-------+
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 2. Increase stock for one product | 3 marks

A. Code: targeted Delta update and stock_qty + 5 logic.

B. Result: changed P102=17 plus the three unchanged products.

Use descriptive filenames such as M1_E02_T02_Code_A.png, M1_E02_T02_Result_A.png, or M1_E02_T02_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

```
Just now (4s)
1  from delta.tables import DeltaTable
2
3  delta_tbl = DeltaTable.forName(spark, "products")
4
5  delta_tbl.update(
6      condition="product_id = 'P102'",
7      set={"stock_qty": "stock_qty + 5"}
8  )
9
10 spark.sql("SELECT * FROM products ORDER BY product_id").show()
See performance (2)
```

Paste screenshot(s) - Result: _____

```
+----------+------------+---------+
|product_id|product_name|stock_qty|
+----------+------------+---------+
|      P101|    Keyboard|       20|
|      P102|       Mouse|       17|
|      P103|     Monitor|        8|
|      P104|     USB Hub|       15|
+----------+------------+---------+
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 3. Inspect transaction history | 2 marks

A. Code/UI: history inspection and named target.

B. Result: UPDATE operation and its actual version number.

Use descriptive filenames such as M1_E02_T03_Code_A.png, M1_E02_T03_Result_A.png, or M1_E02_T03_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code/UI: _____

```
Just now (2s)                                                    5

# A. Inspect full history of the 'products' table
history_df = spark.sql("DESCRIBE HISTORY products")
history_df.select("version", "timestamp", "operation", "operationParameters").show(truncate=False)

# B. Find the version number of the UPDATE operation
update_version = (
    history_df
    .filter("operation = 'UPDATE'")
    .select("version")
    .first()[0]
)
print(f"The UPDATE operation was recorded at version: {update_version}")
```

Paste screenshot(s) - Result: _____

```
history_df:  pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]
+-------+-------------------+------------------------------+--------------------------------
```

```
Just now (1s)                                                    6

# A. Read the baseline version (the version just before the UPDATE)
baseline_version = update_version - 1
print(f"Baseline version for time travel: {baseline_version}")

# B. Time-travel to the baseline version and read the current state
baseline_df = spark.read.format("delta").option("versionAsOf", baseline_version).table("products")
current_df = spark.sql("SELECT * FROM products")

# Get stock_qty for P102 at baseline and current
baseline_qty = baseline_df.filter("product_id = 'P102'").select("stock_qty").first()[0]
current_qty = current_df.filter("product_id = 'P102'").select("stock_qty").first()[0]
difference = current_qty - baseline_qty

print(f"Baseline stock_qty (version {baseline_version}): {baseline_qty}")
print(f"Current stock_qty: {current_qty}")
print(f"Difference: +{difference}")

> See performance (2)
```

Paste screenshot(s) - Result: _____

```
> 2 dataframes: baseline_df, current_df
Baseline version for time travel: 1
Baseline stock_qty (version 1): 12
Current stock_qty: 17
Difference: +5
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Evaluation rubric and evidence map

Task	Criterion	Marks	Evidence
1	Correct external file, typed table and Delta target	2	Code / UI / Result as specified in Task 1
1	Correct baseline rows and captured version	1	Code / UI / Result as specified in Task 1
2	Correct targeted UPDATE expression	2	Code / UI / Result as specified in Task 2
2	Correct current stock after update	1	Code / UI / Result as specified in Task 2
3	History reads correct Delta table	1	Code / UI / Result as specified in Task 3
3	UPDATE version and operation are clearly visible	1	Code / UI / Result as specified in Task 3
4	Correct VERSION AS OF logic using actual baseline	1	Code / UI / Result as specified in Task 4
4	Correct comparison without table mutation	1	Code / UI / Result as specified in Task 4

Total marks: 10. Task count: 4. Screenshot count: variable; provide enough legible evidence to cover every rubric criterion.

Submission requirements

Complete the matching submission template with task-specific screenshot(s) showing every assessed code/configuration and output item. Multiple screenshots per task are allowed and sometimes necessary. Code-only evidence does not prove output; result-only evidence does not prove implementation when the rubric assesses both. Do not submit a generic solution script in place of evidence. Keep your original notebook, Delta objects, pipeline, dashboard and version history accessible. No extra hidden tests or additional report are required.